Documento Explicativo: Arquitectura y Uso de Estructuras de Datos

Proyecto: Sistema Policial de Gestión de Casos e Interrogatorios

Sebastián Chisaba Cruz
Ariadna Valentina Ramírez Santeliz
Jerónimo Pascuas Barroso

21 de octubre de 2026

Resumen

El presente documento detalla la justificación teórica (el *por qué*) y el diseño técnico de implementación (el *cómo*) de cada una de las estructuras de datos empleadas en el desarrollo del **Sistema Policial de Gestión de Casos** en lenguaje C++. Asimismo, se analiza la complejidad algorítmica temporal, la gestión automática de memoria dinámica y se incluye la declaración de transparencia sobre el uso de herramientas de Inteligencia Artificial (45 %) en la documentación y refactorización de clases.

Índice

1. Resumen Arquitectónico del Sistema **2**

2. Análisis Detallado por Estructura de Datos **2**
2.1. Cola con Prioridad Dinámica (`ColaPrioridadCasos` y `NodoCaso`) 2
2.2. Pila Dinámica (`PilaEvidencias` y `NodoPila`) 3
2.3. Cola Circular Estática (`ColaCircularCoartadas`) 4
2.4. Registros (`struct Sospechoso` y `struct Detective`) 4

3. Gestión de Memoria y Prevención de Fugas **5**

4. Declaración de Uso de Inteligencia Artificial **5**

1. Resumen Arquitectónico del Sistema

El sistema modela el flujo operativo de una central de detectives donde los expedientes criminales ingresan en tiempo real, se priorizan según su gravedad, acumulan pistas forenses, se asignan a agentes disponibles y vinculan sospechosos con coartadas rotativas. Para resolver cada requerimiento con eficiencia computacional, se integraron cuatro estructuras de datos fundamentales:

Estructura de Datos	Clase / Entidad en Código	Memoria	Complejidad Principal
Cola con Prioridad	`ColaPrioridadCasos` `NodoCaso`	Dinámica	Inserción: $O(n)$ Atención: $O(1)$
Pila Enlazada (LIFO)	`PilaEvidencias` `NodoPila`	Dinámica	Apilar: $O(1)$ Desapilar: $O(1)$
Cola Circular (FIFO)	`ColaCircularCoartadas`	Estática	Rotación circular: $O(1)$
Registros (`struct`)	`Sospechoso` `Detective`	Estática	Acceso directo: $O(1)$

Tabla 1: Resumen de estructuras de datos y complejidades algorítmicas del proyecto.

2. Análisis Detallado por Estructura de Datos

2.1. Cola con Prioridad Dinámica (`ColaPrioridadCasos` y `NodoCaso`)

¿Por qué se utilizó?

En una central de emergencias policiales, los expedientes criminales no pueden atenderse bajo un esquema estrictamente secuencial de llegada (FIFO simple), dado que la urgencia varía según la naturaleza del delito. Un caso de Prioridad 1 (Alta, ej. *Homicidio*) requiere atención inmediata por encima de un caso de Prioridad 2 (Media) o Prioridad 3 (Baja). Adicionalmente, dado que el sistema cuenta con un generador en tiempo real que inyecta nuevos casos cada 60 segundos, el número total de expedientes activos es indeterminado a priori, haciendo indispensable una estructura dinámica que crezca bajo demanda sin desbordarse.

¿Cómo se implementó?

Se implementó mediante una estructura de nodos enlazados ordenados a través del atributo `prioridad` de la clase `NodoCaso`:

- **Inserción ordenada (`insertarConPrioridad`):** Al instanciar un nuevo nodo en el *Heap* con `new NodoCaso(...)`, se evalúa si la cola está vacía o si la prioridad del nuevo expediente es estrictamente mayor (valor entero menor, p. ej., $1 < 2$) que el nodo en la `cabeza`. De cumplirse, el nuevo nodo se convierte en la nueva `cabeza` en tiempo $O(1)$.
- **Estabilidad FIFO en empates:** Si el nuevo caso tiene menor o igual urgencia que la cabeza, un puntero auxiliar (`actual`) recorre la estructura bajo la condición `actual->sig->prioridad <= prio`. El uso del operador `<=` garantiza que, ante casos de igual prioridad, el recién llegado se ubique detrás de los que ya estaban esperando, preservando el orden cronológico justo.
- **Desencolado (`eliminarPrimero`):** Como la estructura se mantiene ordenada desde la inserción, el caso más urgente siempre reside en `cabeza`. Al emitirse el veredicto, se elimina el primer nodo en tiempo constante $O(1)$ y se libera su memoria mediante `delete`.

2.2. Pila Dinámica (`PilaEvidencias` y `NodoPila`)

¿Por qué se utilizó?

Durante la investigación en la Sala de Evidencias, las pistas se recolectan progresivamente y el detective necesita visualizar en primera instancia el hallazgo más reciente en la parte superior del expediente. Una estructura **LIFO** (*Last In, First Out*) representa de manera exacta el apilamiento físico de reportes forenses. Al implementarse de forma dinámica y anidada dentro de cada `NodoCaso`, cada expediente mantiene su propio historial aislado y solo reserva memoria por la cantidad exacta de evidencias procesadas (entre 0 y 4).

¿Cómo se implementó?

Cada objeto `NodoCaso` contiene como atributo un objeto `PilaEvidencias`, gobernado por el puntero `NodoPila *cabeza`:

- **Operación Push (`apilar`):** Cuando el método `solicitarNuevaEvidencia()` descubre el siguiente rasgo del culpable, invoca `apilar(nuevaPista)`. Se crea un `NodoPila` en memoria dinámica, su puntero `sig` apunta a la `cabeza` actual y `cabeza` se actualiza al nuevo nodo en tiempo $O(1)$.
- **Operación Pop (`desapilar`) y Destructor:** El método `desapilar()` desvincula el nodo superior y ejecuta `delete temp`. Cuando un caso se cierra en la cola de prioridad y se ejecuta `delete aux` sobre el `NodoCaso`, se invoca automáticamente el destructor `~PilaEvidencias()`, el cual vacía toda la pila nodo por nodo evitando fugas de memoria (*memory leaks*).

2.3. Cola Circular Estática (`ColaCircularCoartadas`)

¿Por qué se utilizó?

Cada vez que se genera un nuevo caso criminal, el sistema debe asignar una coartada declarada a cada uno de los 5 sospechosos. Dado que el repertorio de declaraciones consta de un catálogo fijo de 10 coartadas reutilizables entre distintos casos, usar memoria dinámica implicaría reservar y destruir nodos innecesariamente. Una **Cola Circular Estática** permite reciclar el mismo bloque de memoria de forma perpetua en tiempo constante $O(1)$.

¿Cómo se implementó?

La clase `ColaCircularCoartadas` encapsula un contenedor fijo `string elementos[10]` junto con los índices enteros `cabeza`, `ultimo` y `tamano`:

- **Encolado inicial:** El constructor precarga las 10 coartadas avanzando el índice final con aritmética modular: `ultimo = (ultimo + 1) % 10`.
- **Desencolado con rotación (`desencolarYRotar`):** Al requerirse una coartada para un sospechoso, se extrae el elemento en `elementos[cabeza]`, se avanza el frente mediante `cabeza = (cabeza + 1) % 10` e inmediatamente se vuelve a insertar dicho texto al final de la fila (`ultimo = (ultimo + 1) % 10`). De este modo, las coartadas rotan en un ciclo infinito sin agotarse.

2.4. Registros (`struct Sospechoso` y `struct Detective`)

¿Por qué se utilizó?

Tanto los sospechosos como los detectives poseen atributos heterogéneos pero una cantidad fija de integrantes dentro de la lógica de negocio (15 sospechosos en el catálogo global, 5 sospechosos por caso y 4 detectives en la plantilla). El uso de registros (`struct`) permite encapsular todos los atributos de una entidad en una única unidad lógica sin recurrir a variables sueltas, permitiendo acceso inmediato en tiempo $O(1)$.

¿Cómo se implementó?

```
struct Sospechoso {
  string nombre;
  string rasgos[4];
  string coartada;
  bool esCulpable;
};

struct Detective {
  string nombre;
  bool disponible;
  int casosResueltos;
};
```

- **Gestión de Sospechosos:** En el constructor de `NodoCaso`, se selecciona aleatoriamente a 5 sospechosos distintos del catálogo global verificando mediante una tabla de marcas booleanas (`bool yaElegido[15]`) que ningún perfil se repita, copiando cada estructura directamente hacia el expediente y asignándole su coartada desde la cola circular.
- **Gestión de Detectives:** Se instancia la plantilla de 4 estructuras `Detective` en el programa principal y se reutiliza una única función tabular (`mostrarDetectives`) tanto para consultar estadísticas generales como para verificar el estado de disponibilidad antes de asignar un agente al caso prioritario.

3. Gestión de Memoria y Prevención de Fugas

El proyecto garantiza una limpieza total y automática de la memoria dinámica (*Heap*) mediante destructores en dos niveles:

1. **Al cerrar un caso individual:** El método `eliminarPrimero()` ejecuta `delete aux` sobre el `NodoCaso` resuelto, lo que dispara en cascada el destructor `~PilaEvidencias()`, liberando todos los nodos `NodoPila` asociados a ese expediente.
2. **Al salir del programa con casos pendientes:** El destructor `~ColaPrioridadCasos()` itera sobre todos los casos restantes invocando `eliminarPrimero()` hasta vaciar la cola por completo antes de finalizar la ejecución.

4. Declaración de Uso de Inteligencia Artificial

Por transparencia académica, los autores declaran un porcentaje estimado del **45 % de asistencia de herramientas de Inteligencia Artificial** durante el desarrollo del proyecto, distribuido específicamente en las siguientes tareas de apoyo:

- **Documentación en línea y manuales (25 %):** Estructuración de comentarios bajo el estándar Doxygen (`@brief`, `@details`, `@class`, `@struct`), formato de tablas en LaTeX y maquetación del manual de usuario.
- **Refactorización y pulido de clases (20 %):** Unificación de registros (`struct Sospechoso` y `struct Detective`), simplificación del algoritmo de selección aleatoria sin reemplazo y verificación de destructores para prevención de fugas de memoria.

La lógica central del juego, el diseño del flujo de casos criminales, la selección de estructuras de datos y las reglas de negocio del sistema policial fueron definidos y validados por los autores del proyecto.